Topic 17 - Advanced Expressions & PDF Data Extraction

Step 1: Mock ERP Ingestion API Setup (Choose One of 3 Options)

To test ERP order ingestion without access to a live SAP,Oracle NetSuite, or Microsoft Dynamics system:

Option A: Free Cloud Mock API via DummyJSON Custom

Response (Recommended) 1. Open your browser and navigate to **https://dummyjson.com/custom-response**. **2.** In the **JSON Response** editor, paste the mock ERP confirmation payload:

```
{
 "erpStatus": "IMPORTED",
 "erpOrderId": "ERP-992144",
 "system": "SAP S/4HANA Cloud Simulated",
 "statusCode": 200,
 "extractedSource": "PDF_AND_JSON_MULTI_MODAL",
 "message": "Order normalized and posted to General
Ledger successfully" }
```

3. Set **HTTP Status Code** to `200 OK`.
4. Click **Generate API URL** and copy your unique link (e.g., `https://dummyjson.com/c/abc-123`).
5. Use this URL in **Node 8 (`HTTP_PostCleanOrderToERP`)**.

Option B: Postman Mock Server

1. In Postman, create a new Collection named `Enterprise ERP Mocks`.
2. Add a request `POST /erp/orders`.
3. Click **Add Example** with Status Code `200 OK` and body:

```
{
 "erpStatus": "IMPORTED",
 "erpOrderId": "ERP-POSTMAN-5512",
 "importedAt": "2026-10-15T12:00:00.000Z"
}
```

4. Click **Mock Collection** to generate your Postman mock URL: `https://{{mock-id}}.mock.pstmn.io/erp/orders`.
5. Paste this URL into **Node 8**.

Option C: In-n8n Local Webhook Mock Server

1. In n8n, create a workflow named `Mock ERP Server`.
2. Add a **Webhook Node** (`POST /mock/erp/orders`, Mode: Using Respond Node). 3. Connect a **Respond to Webhook Node** returning status `200` with JSON body above.

Step 2A: Simulate Binary PDF Invoice Attachment (`Code_SimulatePDFInvoiceBinary`)

1. **Create Workflow & Manual Trigger:**

   In your n8n dashboard, click **+ Add workflow** in the top-right corner.

   Rename the workflow in the top-left to: `Topic 2 - Advanced Expressions & PDF Data Extraction`.

   Click **+ Add first step...**, search for **`Manual Trigger`**, and add it. Rename to: `Manual_Trigger`.

2. **Add PDF Binary Simulation Node:**

   Hover over the right connector handle of `Manual_Trigger`, click the `+` icon, search for **`Code`**, and add a Code node.

   Double-click the node modal title and rename it to: `Code_SimulatePDFInvoiceBinary`.

3. **Configure Parameters (Parameters Tab):**

   Language: Select **`JavaScript`**.

   Mode: Select **`Run Once for Each Item`**.

   JavaScript Code: Paste the following minimal valid PDF binary generator:

```
// Minimal valid PDF text stream with standard invoice structure
const invoiceText =
`%PDF-1.4
1 0 obj << /Type /Catalog /Pages 2 0 R >> endobj
2 0 obj << /Type /Pages /Kids [3 0 R] /Count 1 >> endobj
3 0 obj << /Type /Page /Parent 2 0 R /MediaBox [0 0 612 792] /Contents 4 0 R /Resources << /Font
<< /F1 5 0 R >> >> >> endobj 4 0 obj << /Length 265 >> stream
BT
/F1 12 Tf
72 700 Td (INVOICE NUMBER: INV-PDF-2026-904) Tj
0 -20 Td (DATE: 2026-10-14) Tj
0 -20 Td (VENDOR: Acme Industrial Automation Corp) Tj
0 -30 Td (ITEM: Industrial Sensor Pack x3 @ 150.00 USD = 450.00 USD) Tj
0 -20 Td (ITEM: DIN Rail Power Supply x2 @ 85.50 USD = 171.00 USD) Tj
0 -30 Td (SUBTOTAL: 621.00 USD) Tj
0 -20 Td (TAX: 49.68 USD) Tj
0 -20 Td (TOTAL: 670.68 USD) Tj
ET
endstream endobj
5 0 obj << /Type /Font /Subtype /Type1 /BaseFont /Helvetica >> endobj
xref
0 6
0000000000 65535 f
0000000009 00000 n
0000000058 00000 n
0000000115 00000 n
0000000244 00000 n
0000000560 00000 n
trailer << /Size 6 /Root 1 0 R >>
startxref
637
%%EOF`;

const base64Data = Buffer.from(invoiceText).toString('base64');

return {
 json: {
 fileName: "vendor_invoice_INV-904.pdf",
 sourceChannel: "Supplier_Portal_Upload",
 fileType: "application/pdf"
 },
```

```
binary: {
data: {
data: base64Data,
mimeType: "application/pdf",
fileName: "vendor_invoice_INV-904.pdf",
fileExtension: "pdf"
}
}
};
```

4. **Test Node Checkpoint:**

   Click **Test step** at the bottom of the modal.

   In the Output pane, switch to the **Binary** tab and verify the `data` binary property exists with `mimeType: application/pdf`.

Step 2B: Simulate Unstructured Legacy JSON Orders (`Code_GenerateLegacyOrders`)

1. **Add Node to Canvas (Parallel Branch):**

   Go back to `Manual_Trigger`. Click its right connector handle and drag a **second** connection out to the canvas below the PDF branch.

   Search for `Code` and place the node.

2. **Rename the Node:**

   Rename to: `Code_GenerateLegacyOrders`.

3. **Configure Parameters (Parameters Tab):**

   Language: Select `JavaScript`.

   Mode: Select `Run Once for All Items`.

   JavaScript Code: Paste the dirty order batch:

```
return [
 {
 json: {
 orderId: "LEG-8841",
 orderDate: "10/14/2026",
 sourceTimezone: "America/New_York",
 customer: {
 rawName: " jOHn dOE-sMITH ",
 email: "JOHN.DOE@GMAIL.COM ",
 phone: "(555) 019-2834"
 },
 items: [
 { sku: "item-01", description: "Mechanical Keyboard", unitPrice: " $129.99
", quantity: 2 },  { sku: "item-02", description: "Wrist Rest", unitPrice:
"24.50", quantity: 1 }
 ],
 discountPercent: 10
 }
 },
 {
 json: {
 orderId: "LEG-8842",
 orderDate: "2026-10-15T08:45:00-07:00",
 sourceTimezone: "America/Los_Angeles",
 customer: {
 rawName: "ALICE WONDERLAND",
 email: "alice@adventures.co.uk",
 phone: "+44 20 7946 0919"
 },
 items: [
 { sku: "item-09", description: "UltraWide Monitor 34-inch", unitPrice: "499.00
USD", quantity: 1 },  { sku: "item-12", description: "Monitor Arm Mount",
unitPrice: "$85.00", quantity: 2 }  ],
 discountPercent: 15
```

```
  }
  }
];
```

4. **Test Node Checkpoint:**

   Click **Test step** and verify 2 raw JSON items appear in the Output table.

Step 3: Extract Text from PDF Binary (`Extract_PDF_Text`)

1. **Add Node to Canvas:**

   Hover over the right connector handle of `Code_SimulatePDFInvoiceBinary`, click `+`, search for **`Extract from File`**, and select the **Extract from File** node (`n8n-nodes-base.extractFromFile`).

2. **Rename the Node:**

   Double-click the header title and rename to: `Extract_PDF_Text`.

3. **Configure Parameters (Parameters Tab):**

   Operation: Click the dropdown and select **`Extract from PDF`** (or `pdf`).

   Binary Property Name: Enter `data` (Mode: **Fixed**).

   Output Format: Select **`Text`** (or leave default to output raw text string into `text` property).

4. **Test Node Checkpoint:**

   Click **Test step** at the bottom of the modal.

   Verify the Output pane converts the binary PDF into a single text property starting with `INVOICE NUMBER: INV-PDF-2026-904`.

Step 4: Parse Raw PDF Text into Structured JSON (`Transform_ParsePDFInvoice`)

1. **Add Node to Canvas:**

   From `Extract_PDF_Text`, click `+`, search for **`Code`**, and add the node.

2. **Rename the Node:**

   Rename to: `Transform_ParsePDFInvoice`.

3. **Configure Parameters (Parameters Tab):**

   Language: Select **`JavaScript`**.

   Mode: Select **`Run Once for Each Item`**.

   JavaScript Code: Paste the regex parsing script:

```
const rawText = $input.item.json.text || "";
```

```
// Extract metadata fields using Regex
const invoiceNumMatch = rawText.match(/INVOICE NUMBER:\s*([A-Z0-9-]+)/i);
const dateMatch = rawText.match(/DATE:\s*([0-9-]+)/i);
const vendorMatch = rawText.match(/VENDOR:\s*([^\r\n]+)/i);
const totalMatch = rawText.match(/TOTAL:\s*([0-9.]+)/i);
const subtotalMatch = rawText.match(/SUBTOTAL:\s*([0-9.]+)/i);
const taxMatch = rawText.match(/TAX:\s*([0-9.]+)/i);

// Extract line items
const itemRegex =
/ITEM:\s*([^x]+)x([0-9]+)\s*@\s*([0-9.]+)\s*USD\s*=\s*([0-9.]+)\s*USD/
gi; const lineItems = [];
let match;
while ((match = itemRegex.exec(rawText)) !== null) {
 lineItems.push({
 sku: "PDF-EXTRACTED-SKU",
 description: match[1].trim(),
 quantity: parseInt(match[2], 10),
 unitPrice: parseFloat(match[3]),
 total: parseFloat(match[4])
 });
}

return {
 json: {
 sourceType: "PDF_INVOICE",
 orderId: invoiceNumMatch ? invoiceNumMatch[1] : "UNKNOWN_INV",
 orderDate: dateMatch ? dateMatch[1] : new Date().toISOString(),
 customer: {
 fullName: vendorMatch ? vendorMatch[1].trim() : "Unknown Vendor",
 email: "billing@acmeindustrial.com",
 phone: "+1-800-555-0199"
 },
 items: lineItems,
 financials: {
 subtotal: subtotalMatch ? parseFloat(subtotalMatch[1]) : 0,
 tax: taxMatch ? parseFloat(taxMatch[1]) : 0,
 total: totalMatch ? parseFloat(totalMatch[1]) : 0
 }
 }
};
```

4. **Test Node Checkpoint:**

   Click **Test step**. Verify structured fields (`orderId: "INV-PDF-2026-904"`, `items: [2 items]`, `financials.total: 670.68`) are created.

Step 5: Normalize Dates with Luxon (`Code_NormalizeDates`)

1. **Add Node to Canvas (JSON Branch):**

   Hover over the right connector handle of `Code_GenerateLegacyOrders`, click `+`, search for `Code`, and add the node.

2. **Rename the Node:**

   Rename to: `Code_NormalizeDates`.

3. **Configure Parameters (Parameters Tab):**

   Language: Select `JavaScript`.

Mode: Select `Run Once for Each Item`.

JavaScript Code: Paste the Luxon timestamp normalizer:

```
const item = $input.item.json;
const rawDate = item.orderDate;
const tz = item.sourceTimezone || 'UTC';

let parsedDate;
if (rawDate.includes('/')) {
 parsedDate = DateTime.fromFormat(rawDate.trim(),
'MM/dd/yyyy', { zone: tz }); } else {
 parsedDate = DateTime.fromISO(rawDate.trim(), { zone: tz });
}

return {
 json: {
 ...item,
 normalizedDateUtc: parsedDate.toUTC().toISO(),
 displayDate: parsedDate.toFormat('yyyy-MM-dd HH:mm:ss')
 }
};
```

4. **Test Node Checkpoint:**

   Click **Test step**. Verify all items now have an ISO-8601 UTC timestamp (`normalizedDateUtc`).

Step 6: Transform Line Items & Calculate Financials (`Transform_ProcessLineItems`)

1. **Add Node to Canvas:**

   From `Code_NormalizeDates`, click `+`, search for `Code`, and rename to: `Transform_ProcessLineItems`.

2. **Configure Parameters (Parameters Tab):**

   Language: Select `JavaScript`.

   Mode: Select `Run Once for Each Item`.

   JavaScript Code: Paste the item financial calculator:

```
const item = $input.item.json;

// 1. Sanitize items and parse currency symbols
const cleanedItems = (item.items || []).map(it => {
 const cleanPriceStr = String(it.unitPrice ||
'0').replace(/[^0-9.]/g, '');  const unitPrice =
parseFloat(cleanPriceStr) || 0;
 const qty = parseInt(it.quantity, 10) || 1;
 return {
 sku: it.sku,
 description: it.description.trim(),
 unitPrice: unitPrice,
 quantity: qty,
 lineTotal: Number((unitPrice * qty).toFixed(2))
```

```
 };
});

// 2. Financial totals
const subtotal = cleanedItems.reduce((acc, curr) => acc +
curr.lineTotal, 0); const discountRate =
(item.discountPercent || 0) / 100;
const discountAmount = Number((subtotal * discountRate).toFixed(2));
const discountedSubtotal = subtotal - discountAmount;
const taxAmount = Number((discountedSubtotal *
0.08).toFixed(2)); // 8% sales tax const grandTotal =
Number((discountedSubtotal + taxAmount).toFixed(2));

// 3. Name Capitalization
const rawName = String(item.customer?.rawName || '').trim();
const cleanedName = rawName.toLowerCase().replace(/\b\w/g, c =>
c.toUpperCase());

return {
 json: {
 sourceType: "LEGACY_JSON",
 orderId: item.orderId,
 orderDate: item.normalizedDateUtc,
 customer: {
 fullName: cleanedName,
 email: (item.customer?.email || "").toLowerCase().trim(),
 phone: item.customer?.phone || ""
 },
 items: cleanedItems,
 financials: {
 subtotal: Number(subtotal.toFixed(2)),
 discount: discountAmount,
 tax: taxAmount,
 total: grandTotal
 }
 }
};
```

3. **Test Node Checkpoint:**

   Click **Test step** and verify clean customer name formatting (`"John Doe-Smith"`) and calculated totals.

Step 7: Standardize ERP Payload with Edit Fields Node (`Edit_Fields_StandardizeERP`)

1. **Add Node & Connect Both Branches:**

   From `Transform_ParsePDFInvoice` (PDF branch), click `+`, search for `Edit Fields` (or `Set`), and select **Edit Fields (Set)**.

   Rename the node to: `Edit_Fields_StandardizeERP`.

   Crucial Connection Step: Now click and drag the output connector handle from `Transform_ProcessLineItems` (JSON branch) and connect it into the **same** input port on `Edit_Fields_StandardizeERP`. Both branches now converge into this standardization node!

2. **Configure Parameters (Parameters Tab):**

   Mode: Select `Manual Mapping`.

   Include Other Input Fields: Switch the toggle to `OFF` (ensures dirty source-specific fields are discarded).

   Click **+ Add Field** for each canonical field below:

   1. **Field Name:** `documentType` | **Type:** `String` | Mode: **Expression (`fx`)** | **Value:** `={{ $json.sourceType }}`
   2. **Field Name:** `erpTransactionId` | **Type:** `String` | Mode: **Expression (`fx`)** | **Value:** `={{ 'TX-' + $json.orderId }}`
   3. **Field Name:** `accountName` | **Type:** `String` | Mode: **Expression (`fx`)** | **Value:** `={{ $json.customer.fullName }}`
   4. **Field Name:** `contactEmail` | **Type:** `String` | Mode: **Expression (`fx`)** | **Value:** `={{ $json.customer.email }}`
   5. **Field Name:** `lineItemCount` | **Type:** `Number` | Mode: **Expression (`fx`)** | **Value:** `={{ $json.items.length }}`
   6. **Field Name:** `amountPayable` | **Type:** `Number` | Mode: **Expression (`fx`)** | **Value:** `={{ $json.financials.total }}`
   7. **Field Name:** `processedAt` | **Type:** `String` | Mode: **Expression (`fx`)** | **Value:** `={{ $now.toISO() }}`

3. **Test Node Checkpoint:**

   Click **Test step**. Verify all items from both sources output clean, identical schema fields.

Step 8: Post to Enterprise Mock ERP (`HTTP_PostCleanOrderToERP`)

1. **Add Node to Canvas:**

   From `Edit_Fields_StandardizeERP`, click `+`, search for `HTTP Request`, and add the node. Rename to: `HTTP_PostCleanOrderToERP`.

2. **Configure Parameters (Parameters Tab):**

   Method: Select `POST`.

   URL: Toggleto **Expression (`fx`)**, enter:

   DummyJSON Option A: `https://dummyjson.com/c/<your-id>`

   Postman Option B: `https://{{mock-id}}.mock.pstmn.io/erp/orders`

   Local Option C: `http://localhost:5678/webhook/mock/erp/orders`

   Authentication: Select `None`.

   Send Body: Switch toggle to `ON`.

Body Content Type: Select `JSON`.

Specify Body: Select `Using JSON`.

JSON: Switch to **Expression (`fx`)** and enter:

```
={{ $json }}
```

3. **Configure Settings (Settings Tab):**

   Click the **Settings** tab at the top.

   Toggle**Retry on Fail** to `ON`.

   Max Tries: `3`.

   Wait Between Tries (ms): `3000`.

4. **Test Node Checkpoint:**

   Click **Test step**. Verify that the mock ERP returns HTTP status `200` with `erpStatus: "IMPORTED"`.

Step 9: Audit Summary Node (`Code_AuditSummary`)

1. **Add Node to Canvas:**

   From `HTTP_PostCleanOrderToERP`, click `+`, search for `Code`, and add the node. Rename to: `Code_AuditSummary`.

2. **Configure Parameters (Parameters Tab):**

   Language: Select `JavaScript`.

   Mode: Select `Run Once for All Items`.

   JavaScript Code: Paste the batch compliance aggregation script:

```
const items = $input.all();
const totalRevenue = items.reduce((sum, it) => sum + (it.json.amountPayable || 0),
0);

return [{
 json: {
 batchStatus: "COMPLETED",
 totalRecordsProcessed: items.length,
 totalRevenueProcessed: Number(totalRevenue.toFixed(2)),
 summaryTimestamp: new Date().toISOString()
 }
}];
```

3. **Test Node Checkpoint:**

   Click **Test step**. Verify the single consolidated audit record displays `batchStatus: "COMPLETED"` and total records processed.